Azure Sentinel SOC Home Lab

A hybrid, on-premises-to-cloud SOC lab built to practice detection engineering and incident response with Microsoft Sentinel. A locally virtualized Windows Server domain controller is connected to Azure via Azure Arc, with security event data flowing into a Log Analytics workspace and Microsoft Sentinel for analysis.

Why hybrid, not a native Azure VM

Most beginner Sentinel labs spin up a Windows VM directly inside Azure. This lab instead runs the domain controller locally in VirtualBox and connects it to Azure through Azure Arc, which is closer to how many real organizations onboard on-premises or non-Azure infrastructure into a cloud SIEM. It also demonstrates the Arc registration, authentication, and agent troubleshooting layer that a pure Azure-VM lab skips entirely.

Architecture

- **Windows Server 2025 (VirtualBox VM)** — domain controller for `soclab.local`, the log source for this lab
- **Kali Linux (VirtualBox VM)** — attacker/simulation host, sharing a VirtualBox NAT Network with the domain controller so both reach each other and the internet independently
- **Azure Arc** — registers the on-premises domain controller as a manageable resource in Azure
- **Azure Monitor Agent (AMA)** — installed via Arc extension, reads Windows Event Logs on the domain controller
- **Data Collection Rule (DCR)** — defines exactly which events AMA collects and forwards
- **Log Analytics Workspace** — central store for all collected log data
- **Microsoft Sentinel** — SIEM layer on top of the workspace, used for querying, detection rules, and incident management

[Kali Linux VM] --(attacks)--> [Windows Server 2025 DC]

|

Azure Arc agent

|

Azure Monitor Agent (AMA)

|

Data Collection Rule

|

Log Analytics Workspace

|

Microsoft Sentinel

Data sources collected

Two Windows Event Log subscriptions are configured on the Data Collection Rule, each targeting a specific log with a custom XPath filter rather than collecting entire log categories wholesale.

Security log:

Security!*[System[(EventID=4624 or EventID=4625 or EventID=4688 or EventID=4720 or EventID=4732 or EventID=4698)]]

PowerShell Operational log:

Microsoft-Windows-PowerShell/Operational!*[System[(EventID=4104)]]

Event ID	**Log**	**Meaning**
4624	Security	Successful logon
4625	Security	Failed logon attempt

Event ID	Log	Meaning
4688	Security	Process creation (with command-line auditing enabled)
4720	Security	User account created
4732	Security	Member added to a security-enabled local group
4698	Security	Scheduled task created
4104	PowerShell Operational	Script block logging (captures deobfuscated PowerShell content, not just the launch of powershell.exe)

These six event types were chosen to cover the core arc of a typical intrusion: initial access attempts (4624/4625), what an attacker does once inside (4688, 4104), and common persistence or privilege escalation actions (4698, 4720, 4732), rather than collecting the entire Security log, which would be noisy and, in a production environment, costly.

Prerequisites enabled on the domain controller

By default, a fresh Windows Server install does not audit everything needed for the above event IDs to actually fire. The following were configured before data would flow:

- `auditpol /set /subcategory:"Process Creation" /success:enable` — required for 4688 to generate at all
- `auditpol /set /subcategory:"Other Object Access Events" /success:enable /failure:enable` — required for 4698
- Local Group Policy: **Computer Configuration > Administrative Templates > System > Audit Process Creation > Include command line in process creation events** — Enabled (without this, 4688 fires but the CommandLine field is blank)
- Local Group Policy: **Computer Configuration > Administrative Templates > Windows Components > Windows PowerShell > Turn on PowerShell Script Block Logging** — Enabled (source of event 4104)

Logon/logoff and account management auditing were already enabled by default on a fresh domain controller promotion.

Current status

The pipeline is confirmed working end to end. Test events for 4624 and 4688 have been generated, ingested, and verified with KQL queries directly against the `SecurityEvent` table in the Sentinel-enabled Log Analytics workspace.

Roadmap

- [ ] Formalize 3–5 attack simulation scenarios: RDP brute force, PowerShell encoded command execution, new local admin account creation, scheduled task persistence
- [ ] Write custom KQL detection logic for each scenario and convert into Sentinel scheduled analytics rules
- [ ] Trigger each scenario and practice the full incident lifecycle in Sentinel: triage, investigation, closure with documented findings
- [ ] **Phase 2 (planned):** AI-augmented incident triage — export Sentinel incidents via the REST API and feed them to an LLM to generate plain-English incident summaries, MITRE ATT&CK technique mapping, remediation recommendations, and executive-level reports

Troubleshooting journal

Building this lab surfaced several non-obvious issues, from a corrupted detection query that failed silently, to domain controller clock drift breaking Azure authentication. A full writeup is in [`docs/troubleshooting-journal.md`](docs/troubleshooting-journal.md).